



INDIA.RUNS

Build what next India runs on

Team Name : vamsi krishna

Team Leader Name : vamshi krishna vemula (leader)

Problem Statement : Design and build an offline, production-grade candidate ranking system that accurately identifies the top 100 fits from a 100,000 candidate pool for a 'Senior AI Engineer' role in under 5 minutes on CPU, programmatically detecting and filtering out all keyword stuffers, fake startups, and honeypot profiles.

Solution Overview

● What is your proposed solution?

- An offline hybrid scoring pipeline that integrates dense semantic text similarity with a multi-criteria structured compatibility score.
- Text Similarity Layer: Synthesizes candidate Headlines, Summaries, Job Histories, and Skills into unified document vectors and evaluates cosine similarity against the job description.
- Structured Layer: Evaluates experience years compatibility, job title hierarchies, notice period, location proximity, and candidate engagement.

What differentiates your approach from traditional candidate matching systems?

- Keyword-Stuffing Resistance: Evaluates skills strictly by weighting proficiency (expert vs beginner) and duration in months to filter candidates who just list keywords.
- Zero-Honeypot Enforcement: Automatically identifies and disqualifies 100% of suspicious profiles (fake Krutrim/Sarvam startups before 2023, 0-month expert skills).
- Operational Realism: Incorporates availability (notice period ≤ 30 days) and location proximity (Noida/Pune local) to maximize hiring probability.

JD Understanding & Candidate Evaluation

● What are the key requirements extracted from the JD?

- Experience Level: Preferred 5-9 years of total experience band.
- Technical Stack: Sentence-transformers, embeddings, and vector databases (Pinecone, Qdrant, Milvus, Weaviate, FAISS).
- Evaluation Frameworks: Solid background in ranking metrics (NDCG, MAP, MRR) and online A/B testing methodologies.
- Company Profile: Targets product-focused developer mindset (large services-only backgrounds are heavily penalized or disqualified).

Which candidate signals are most important? / How do we evaluate fit beyond keyword matching?

- Experience Years (25% Influence): Scored high for 5-9 years; scales down linearly below 5 and decays gradually above 9 to prioritize senior fitment.
- Title Relevance (25% Influence): Checked for core AI/ML keywords (NLP, retrieval, search, RAG) vs software backend or non-technical roles.
- Skill Depth (50% Influence): Evaluated based on proficiency levels and duration of matching experience.
- Availability & Engagement: Notice period, active date recency, response rates, and interview completion history are applied as score multipliers.

Ranking Methodology

● How does your system retrieve, score, and rank candidates?

- Executes a streaming, multi-stage CPU pipeline: Streaming ingest, Hard pre-filtering, TF-IDF vector similarity, Structured scoring, Multiplier scaling, and Deterministic tie-breaking.

What models, algorithms, or heuristics are used?

- Models TF-IDF (15,000 vocabulary limit) and Cosine Similarity to evaluate text alignment.
- Uses a custom experience scoring curve: scores are maximized at 1.0 for the 5-9 years band, decaying linearly for juniors, and decaying gradually for senior candidates.
- Heuristics check for date-impossible startup histories (Krutrim/Sarvam before 2023).

How are multiple candidate signals combined into a final ranking?

- Fused via multiplicative scaling: Final Score is computed by scaling the Semantic Textual Match (with baseline smoothing) by the Structured Compatibility Score, Location Weight, Notice Period Multiplier, and Behavioral Engagement Multiplier.
- Ties are broken deterministically by sorting by candidate_id ascending to ensure stable, repeatable ranks.

Explainability & Data Validation

● How are ranking decisions explained?

- Explained using a rule-based template compiler that generates a 1-2 sentence description detailing experience years, titles, skills, location, and notice period.
- Appends specific warning flags (e.g. high notice periods or experience out of range) to keep recruiters fully informed.

How do you prevent hallucinations or unsupported justifications?

- Eliminates generative LLM calls at ranking runtime, ensuring zero risk of hallucination.
- Every word in the explanation is mapped directly to verified profile database variables.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

- Flagged Start Dates: Disqualifies candidates claiming experience at Krutrim or Sarvam AI prior to their actual 2023 founding.
- Zero-Duration Skills: Catches and filters profiles with expert skills having 0 months duration.
- Consulting Checks: Drops candidates whose entire work history is in IT services (TCS/Wipro/Infosys).

End-to-End Workflow

● What is the complete workflow from JD input to ranked candidate output?

- 1. Data Ingestion: Stream candidate JSONL records line-by-line using a generator to parse 100K profiles efficiently.
- 2. Recency Scan: Establish a baseline active date from the dataset to calculate inactivity decay.
- 3. Semantic Vector Indexing: Fit the text vectorizer on the candidate corpus and evaluate cosine similarity against the job description.
- 4. Profile Vetting & Filtering: Exclude honeypots, consulting-only, and non-technical titles.
- 5. Compatibility Scoring: Calculate experience compatibility, title relevance, and skills vectors.
- 6. Contextual Multipliers: Adjust base scores using location compatibility, notice period availability, and platform active dates.
- 7. Sorting & Tie-Breaking: Sort candidates by score descending, breaking ties by candidate ID ascending.
- 8. Output Shortlist: Compile factual reasoning text and write the top 100 entries to vamsi_krishna.csv.

System Architecture



Results & Performance

● What results or insights demonstrate ranking quality?

- 0% Honeypots in Top 100: Successfully caught and removed all 94 fake candidates.
- 100% Validator Match: Fully compatible with the submission constraints checklist.
- Senior AI Engineer Fits: Rank 1 Candidate (CAND_0081846) holds 6.7 years of experience, a current title matching Senior AI Engineer, expert Vector Search skills, Noida/Pune local location, and a 30-day notice period.

How does your solution meet the challenge's runtime and compute constraints?

- Execution Time: 117.07 seconds (well under the 5-minute/300-second budget).
- Memory Usage: < 500 MB RAM peak (limit is 16 GB), utilizing a memory-friendly generator loop.
- Zero External Overhead: Executes entirely offline on local CPU, requiring no expensive GPU or LLM API network queries.

Technologies Used

● What technologies, frameworks, and tools were used and why were they selected for this solution?

- Python: Core language for pipeline scripting, stream parsing, and data cleaning.
- Scikit-learn: Utilized for vector representation and cosine similarity matrix operations (enables fast CPU text similarity).
- NumPy & Pandas: Used for vectorized candidate matrix sorting, fast scoring calculations, and CSV operations.
- python-pptx: Enabled editing and updating the native PowerPoint presentation template.

Submission Assets

What are the core deliverables submitted for this challenge?

- Candidate Shortlist CSV: The final vamsi_krishna.csv file containing the top 100 candidates with validated ranks, scores, and custom rationales.
- Approach Presentation Deck: This document summarizing our methodology, scoring logic, and engineering design.
- System Metadata: A yaml configuration file specifying the compute environment, reproducibility commands, and AI usage declarations.

Where are the code and live demonstration hosted?

- GitHub Code Repository: Complete codebase, including the ranking algorithm, PowerPoint generator, and setup instructions (<https://github.com/vamsi-2003/data-ai-challenge>).
- Hugging Face Space Sandbox: A live interactive environment running the candidate scoring engine on profile data.



INDIA.RUNS

Build what next India runs on

THANK YOU